

TECHNICAL DOCUMENTATION · v1.0

ReeVue

A community-driven tech career review platform where engineers discover, share, and evaluate real interview experiences and workplace insights.

React 19 · Firebase 12 · Vite 8 · Bootstrap 5 · React Router 7 · Cloud Firestore

1 Project Overview

ReeVue is a web application enabling engineers and tech professionals to discover, share, and evaluate interview experiences and job reviews at major technology companies. The platform functions as a community-driven resource that helps users make informed career decisions based on authentic peer experiences.



Core idea: Aggregated, anonymous feedback from real candidates — giving job seekers an honest picture before they walk into an interview room.

Core Features

- **Anonymous interview feedback** from verified candidates at top tech companies
- **Aggregated company ratings** with difficulty assessments for full interview loops
- **Community Q&A forum** for technical and behavioral interview preparation
- **Personalized profiles** tracking user contributions, badges, and saved companies

Target Audience

- Computer science students preparing for tech industry interviews
- Software engineers evaluating career opportunities at new companies
- Tech professionals looking to share and document workplace experiences

2 Technology Stack

Frontend

Technology	Version	Purpose
React	^19.2.6	Core UI framework with functional components and hooks
React DOM	^19.2.6	Renders React component trees to the DOM
React Router DOM	^7.18.0	Client-side routing and navigation management
Vite	^8.0.12	Build tool and dev server with hot module replacement

Backend & Database

Service	Version	Usage
Firebase	^12.15.0	Backend-as-a-Service platform
Firebase Auth	included	Email/password user authentication
Cloud Firestore	included	NoSQL database for reviews with real-time sync
Firebase Analytics	included	User engagement and usage tracking

UI & Styling

Library	Version	Role
Bootstrap	^5.3.8	Responsive grid system and utility classes
Bootstrap Icons	^1.13.1	Icon library used throughout the UI
SweetAlert2	^11.26.25	Styled modal dialogs replacing native browser alerts
custom.css	—	Teal brand theme, dark mode, and animations

3 Project Structure

The project follows a standard Vite + React architecture with clear separation of concerns across pages, components, context, and Firebase configuration.

```
DEPI_REACT-main/
├─ public/           # Static assets (hero images, icons, favicon)
├─ src/
│  └─ assets/        # Images bundled by Vite
```

```
| | components/      # Reusable UI components
| | context/        # React Context providers
| | firebase/       # Firebase configuration & exports
| | pages/          # Full-page route components
| | App.jsx         # Root component + routing
| | App.css         # Base styles
| | custom.css      # Brand theme overrides
| | main.jsx        # Application entry point
| | index.html      # HTML shell template
| | package.json    # Dependencies manifest
| | vite.config.js  # Vite build configuration
```

Components Directory

Component	Description
Navbar.jsx	Top navigation with search, active tab highlighting, user menu, and theme toggle
Footer.jsx	Site-wide footer with links and branding
ReviewCard.jsx	Single review display with rating, pros/cons, difficulty badge, and edit/delete actions
StarRating.jsx	Dual-mode star component — interactive (input) and display (read-only)
CompanyDetails.jsx	Company profile body with aggregated stats and filtered review list
SubmitReview.jsx	Inner wizard component (also has a full-page version in /pages)
Login.jsx	Compact login form component (also has a full-page version in /pages)
Profile.jsx	User profile card sub-component

4 Application Routing

ReeVue uses React Router DOM v7 with BrowserRouter. Routes are declared in App.jsx and split into public and protected sections via a ProtectedRoute guard component.

Public Routes

Path	Component	Description
/	Home	Landing page with hero, company directory, and featured reviews
/reviews	Reviews	All reviews with filter and sort controls
/search	Search	Search results page, receives ?q= query param

Path	Component	Description
/company/:name	CompanyProfile	Per-company profile with aggregated stats and filtered reviews
/login	Login	Email/password sign-in form
/signup	SignUp	Registration form with password strength meter

Protected Routes

These routes require an authenticated user. Unauthenticated visitors are redirected to /login via the ProtectedRoute component.

Path	Component	Description
/submit-review	SubmitReview	5-step wizard to submit or edit a review
/profile	UserProfile	User dashboard with reviews, saved companies, Q&A, and settings
/savedcompanies	SavedCompanies	Bookmarked companies list
/qa	InterviewQA	Community interview Q&A forum

5 Context & State Management

ReeVue uses the React Context API for global state. Three context providers wrap the app from the outside in — no third-party state library required.

ThemeContext

Export	Type	Description
isDark	boolean	Current theme state
toggleTheme()	void	Switches and persists theme; applies .dark to document.body
useTheme()	hook	Exported hook for consuming the theme context

AuthContext

Property / Method	Type	Description
user	Object null	Current user: { name, email, bio, uid }
isLoggedIn	boolean	Derived from !!user
login(email, pw)	Promise	Calls Firebase signInWithEmailAndPassword

Property / Method	Type	Description
signup(email, pw, name)	Promise	Creates Firebase account + sets displayName
logout()	Promise	Signs out from Firebase, clears localStorage
updateProfile(payload)	Promise	Updates local state and Firebase display name



Note: window.alert is globally overridden with SweetAlert2 modals. The app renders null (a loading screen) until both authLoading and dbLoading resolve to false.

CompaniesContext

Export	Type	Description
searchFilters	Object	{ industry, city, companySize, starRating, difficulty } — all default to 'All' / null
setFilters(payload)	dispatch	Dispatches SET_FILTERS to merge partial filter updates
resetFilters()	dispatch	Dispatches RESET_FILTERS to restore initial state

6 Firebase Integration

Firebase is configured in src/firebase/config.js and exports three services consumed across the application.

Exported Services

Export	Description
auth	Firebase Authentication instance
db	Cloud Firestore database instance
app	Firebase app instance
analytics	Firebase Analytics instance

Firestore Collections

Collection	Key Fields	Description
reviews	companyName, jobTitle, rating, difficulty, pros, cons, authorEmail, recommend, isAnonymous, createdAt, helpfulCount	Primary data store. Queried in real-time via onSnapshot in AuthContext. New documents added via addDoc with serverTimestamp.

Firestore Operations

- **CRUD:** `getDocs`, `addDoc`, `updateDoc`, `deleteDoc`, `getDoc`
- **Auth:** `onAuthStateChanged`, `signInWithEmailAndPassword`, `createUserWithEmailAndPassword`, `signOut`, `updateProfile`
- **Real-time:** `onSnapshot` listener on the reviews collection
- **Timestamps:** `serverTimestamp` for consistent server-side review ordering

7 Pages & Components

Home Page /

The landing page is composed of three distinct sections:

- **Hero Section:** Full-viewport background image with light/dark variant, headline, and CTAs. Background switches based on `isDark` from `ThemeContext`.
- **Company Directory:** Dynamically built from Firestore review data. Groups reviews by company, calculates average rating and modal interview difficulty. Each card navigates to `/company/:name`.
- **Highest Rated Reviews:** Fetches all reviews, sorts by rating descending, displays the top 2 using `ReviewCard`.

Reviews Page /reviews

A filterable, sortable list of all reviews synced from Firestore into localStorage.

- Filter by interview difficulty: All, Easy, Medium, Hard
- Sort by: Most Helpful, Highest Rated, Newest First
- Text search across `companyName`, `jobTitle`, `pros`, `cons` fields
- Review owners see Edit and Delete action buttons on their `ReviewCard` components

Submit Review /submit-review [Protected]

A 5-step guided wizard for submitting or editing a review:

Step	Title	Fields
1	Company Profile	Company Name, Industry Sector
2	Role Specifications	Job Title, Employment Type, Location City
3	Workspace Insights	Pros (15+ chars), Cons (15+ chars)
4	Experience Rating	Star Rating (1–5), Difficulty (Easy / Medium / Hard), Recommend toggle
5	Review & Publish	Live preview, anonymous posting toggle, submit or save draft

- ✓ Drafts auto-save to `reevue_review_draft` in `localStorage`. Edit mode loads existing review data from Firestore by `editingReviewId` passed via router state.

Company Profile `/company/:name`

Reads the company name from URL params, loads all reviews from `localStorage`, then renders `CompanyDetails` — which handles: aggregated rating summary, difficulty breakdown, per-review list filtered to that company, and a Write Review CTA.

Search `/search`

Receives a query string via `?q=` URL parameter. Filters reviews across `companyName`, `jobTitle`, `pros`, and `cons` fields. Also computes matching company cards with aggregated stats. Authenticated users can edit/delete their own reviews inline.

Interview Q&A `/qa` [Protected]

- Questions stored in `localStorage` (`reevue_qa_v1`) with seed data for Google, Stripe, Meta, Amazon
- Department filter: All, Engineering, Product, Design, Operations, Other
- Accordion-style expand/collapse per question
- Add Question form: company name, department, and question text (10+ chars)
- Inline answer input under each expanded question

User Profile `/profile` [Protected]

A four-tab personal dashboard:

- **My Reviews:** All reviews authored by the logged-in user. Supports inline edit and delete.
- **Saved Companies:** Bookmarked companies loaded from `reevue_saved_companies`.
- **My Q&A:** Q&A questions submitted by the user or with community answers.
- **Settings:** Name and bio edits synced to `AuthContext` + `Firebase`. Notification preferences (UI only).

Achievement badges are dynamically awarded based on review count and helpful vote thresholds.

Login & Sign Up `/login` `/signup`

Both pages hide the `Navbar` and `Footer` via an `isLoginPage` check in `App.jsx`. Login performs email format and 8-character password validation before calling `Firebase`. Sign Up includes a password strength meter (Weak / Fair / Good / Strong) and optional phone and bio fields.

8 Data Flow & Storage

localStorage Keys

Key	Type	Content
reevue_auth_v1	JSON	{ user: { name, email, bio, uid } } — persisted login session
reevue_reviews_v1	JSON Array	All reviews mirrored from Firestore for fast local reads
reevue_review_draft	JSON	In-progress review wizard state for draft recovery
reevue_qa_v1	JSON Array	Interview Q&A entries (local-only, no Firestore sync)
reevue_theme	String	'dark' or 'light' — persisted theme preference
reevue_saved_companies	JSON Array	User-bookmarked company objects (local-only)

Reviews Data Flow

Step	Action	Detail
1	User submits review form	addDoc() writes new document to Firestore reviews collection
2	AuthContext onSnapshot fires	Updates latestReviewsRef and writes to reeview_reviews_v1 in localStorage
3	Pages read from localStorage	All components read synchronously for fast access without waiting on Firestore
4	Deletions cascade to Firestore	localStorage.setItem is intercepted, compared with Firestore mirror, then deleteDoc is called on missing items

Auth Data Flow

Step	Action	Detail
1	User logs in	Firebase Auth → onAuthStateChanged fires → formats user object and persists to localStorage
2	App loads	loadAuthState() reads localStorage immediately for zero-flash auth persistence
3	User logs out	Removes localStorage entry → calls Firebase signOut() → clears user state

9 Authentication System

ReeVue implements a full Firebase Authentication flow with persistent sessions. The ProtectedRoute guard ensures all sensitive routes redirect unauthenticated visitors.

Sign Up Flow

- User fills in first name, last name, email, optional phone, password (8+), confirm password, optional bio
- Client-side validation runs before any network call
- `createUserWithEmailAndPassword` creates the Firebase account
- `updateProfile` sets the `displayName` on the Firebase Auth user
- `onAuthStateChanged` fires, formats the user object, persists to `localStorage`
- User is redirected to the home page

Login Flow

- Email format and password length validated client-side before any network call
- `signInWithEmailAndPassword` called with Firebase Auth
- On success, `onAuthStateChanged` updates context state
- User is redirected to the home page

Protected Route Guard

ProtectedRoute is an Outlet wrapper that reads `isLoggedIn` from `AuthContext`. If false, it renders `<Navigate to="/login" replace />`. All four protected routes are nested under a single `ProtectedRoute` element in the routes tree.

Logout

- Removes `reevue_auth_v1` from `localStorage`
- Sets `user` state to `null`
- Calls Firebase `signOut()`
- Navigates to the home page

10 Getting Started

Prerequisites

- Node.js 18+ and npm
- A Firebase project with Authentication (Email/Password) and Firestore enabled

Installation

```
# 1. Clone or extract the project
cd DEPI_REACT-main
```

```
# 2. Install dependencies
npm install

# 3. Configure Firebase
#   Edit src/firebase/config.js with your Firebase project credentials

# 4. Start development server
npm run dev

# 5. Build for production
npm run build
```

Available Scripts

Command	Description
npm run dev	Start Vite development server with hot module replacement
npm run build	Build optimized production bundle to the /dist directory
npm run preview	Preview the production build locally before deployment
npm run lint	Run ESLint with React hooks and refresh plugin rules

Firebase Setup Checklist

- Create a Firebase project at console.firebase.google.com
- Enable **Email/Password** authentication in the Firebase console
- Create a **Firestore** database and set appropriate security rules
- Copy your Firebase config values to `src/firebase/config.js`
- (Optional) Enable Firebase Analytics for usage tracking